Backups and Restore Testing

a backup you never restored is a hope

Watch the 4-min video

In one line: keep full, incremental and log-based backups that give point-in-time recovery, store immutable copies in another region and account, and prove them by restoring automatically and regularly, because a backup only counts once you've restored it within your RTO.

1 Three years of backups, none usable

A company backed up its database every night for three years. Ransomware hits. They go to restore: the backups are empty, corrupted, or encrypted too, because the attacker had the same admin access. Nobody had ever tried a restore. A backup you have never restored is a hope, not a recovery plan.

2 Kinds of backup

Kind	Copies	Restore	Trade-off
Full	everything	one step	slow and large
Incremental	changes since the last backup	full + every incremental, in order	small and fast; long fragile chain
Snapshot	a disk or volume at a point in time (copy-on-write)	create a volume / instance from it	cheap and quick; often same account and region
Log archive (PITR)	base backup + continuous transaction log	restore base, replay log to any second	finest RPO; needs the whole log chain

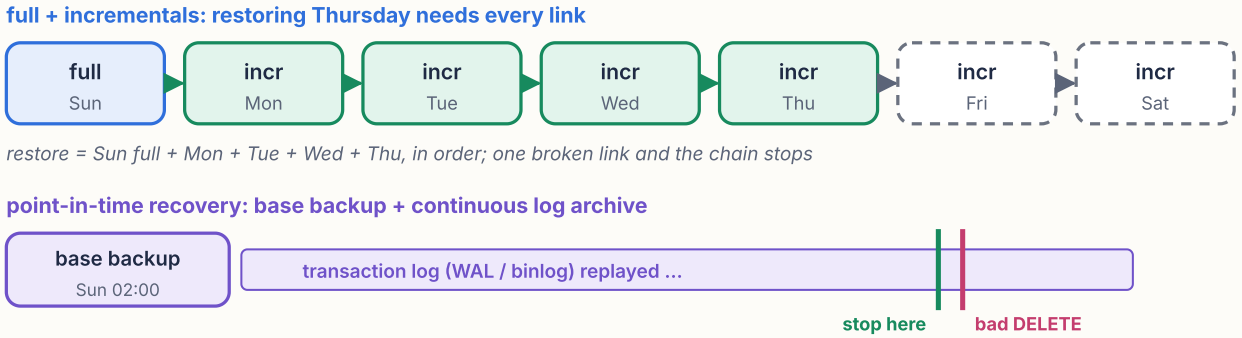


Figure 1. Top: an incremental chain. Bottom: point-in-time recovery replays the log up to just before the mistake.

Point-in-time recovery (PostgreSQL WAL archiving, MySQL binlogs, managed PITR in RDS or Cloud SQL) turns a daily recovery point into a per-second one: restore the base backup, then replay the log up to one second before someone ran the bad `DELETE` (lesson 13).

3 Backups must survive what production doesn't

- **3-2-1 rule:** three copies, on two kinds of media, one off-site. In the cloud: another region *and* another account.

- **Separate account:** one compromised admin key must not be able to delete production and backups together.
- **Immutable:** S3 Object Lock in compliance mode (or Azure immutable blob storage) means nobody, not even the root user or an attacker, can delete or change a backup before retention ends.
- **Keys:** back up and test access to the encryption keys; an encrypted backup without its key is noise.

4 Restore testing

Restores fail for hidden reasons: missing encryption keys, a changed schema, an expired IAM permission, or a backup job that silently stopped weeks ago. Restore **time** matters too: a 10 TB restore may take far longer than your RTO. The only proof is a real restore, done regularly and automatically.

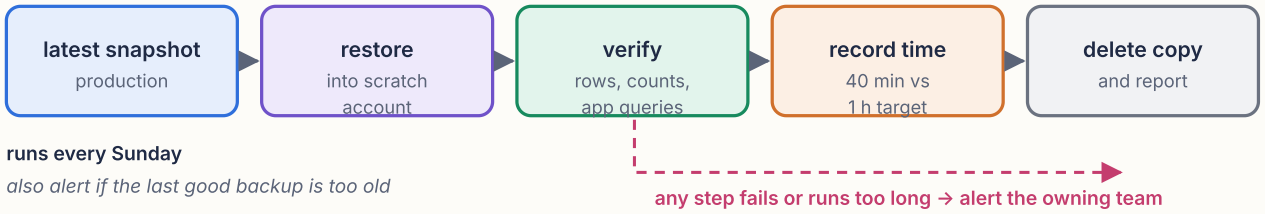


Figure 2. An automated weekly restore test. It measures restore time against the RTO and fails loudly.

WORKED EXAMPLE: THE SUNDAY RESTORE PIPELINE

Every Sunday a pipeline restores the latest production snapshot into a **scratch account**. It checks that the orders table has today's rows, compares row counts with production, and runs a few read-only queries from the app. It records the restore time, currently **40 minutes** against a **1-hour** target, then deletes the copy. One week the restore failed: a **key rotation** had broken access to older snapshots. They fixed it months before a real disaster would have found it.

COMMON MISTAKES

- Backups in the same account and region as production.
- Monitoring that the backup job ran, not that its output restores.
- Never measuring restore time, then discovering it's ten times the RTO.
- Treating replicas as backups: replication copies deletes and corruption instantly.

5 Key terms

Full backup: complete copy of the data

Incremental backup: only what changed since the last backup

Snapshot: point-in-time image of a volume, usually copy-on-write

PITR: base backup + log replay to any moment

3-2-1 rule: 3 copies, 2 media, 1 off-site

Object lock: write-once storage: no delete before retention ends

6 Interview questions

> **Q1. Full vs incremental backup?** BASIC
> A full backup copies everything and restores in one step; an incremental copies only changes and needs the full plus every incremental in order to restore.

Q2. What is point-in-time recovery? BASIC

Restoring a base backup and replaying the archived transaction log up to a chosen moment, e.g. just before a bad delete.

Q3. How do you protect backups from ransomware? SCENARIO

Keep immutable copies (object lock) in a separate account and region, with tight, separate access, so a compromised production admin can't touch them.

Q4. Design restore testing for a critical database. SCENARIO

An automated, scheduled restore into an isolated account, data checks (counts, recent rows, app queries), restore time recorded against the RTO, alerts on failure, then cleanup.

Q5. Why is a backup not a backup until restored? TRICKY

Restores fail for hidden reasons (missing keys, changed schemas, expired permissions, silently stopped jobs) and only a real restore proves the data is usable and the time fits your RTO.

Q6. We have synchronous replicas in three zones. Do we need backups? TRICKY

Yes: replicas protect against hardware and zone loss, but they replicate bad writes, deletes and ransomware encryption instantly.

Next: SD-K-015 · Multi-Region Architecture backendroadmap.com